FUNDWISE

Exact Budget. Every Plan. Smallest Team.

ALGORITHM SUBMISSION · TECHNICAL DOCUMENT

A practical funding-plan optimizer powered by a recursive Sum of Subsets search. This document describes the exact algorithm, its correctness, complexity, safe pruning rules, and verification cases.

The generated standalone source file is **CODE.py**. The official first-page submission checklist was not provided; no checklist requirements have been assumed.

1. Problem statement

Given n positive request costs in their original order and an integer target budget T, count every subset whose costs sum exactly to T. Also return the smallest number of selected requests, the number of subsets with that minimum size, and the lexicographically smallest increasing 1-based index list among minimum-size subsets.

2. Input and output specification

Input is whitespace-separated: n and T, followed by exactly n positive integer costs. Constraints: $0 \le n \le 22$, $0 \le T \le 1{,}000{,}000$, and $1 \le \text{cost} \le 1{,}000{,}000$. Output uses four lines: COUNT, MIN_SIZE, OPTIMAL_COUNT, and CHOSEN. CHOSEN is EMPTY for the empty subset and NONE when no plan exists.

```
COUNT 4
MIN_SIZE 1
OPTIMAL_COUNT 2
CHOSEN 3
```

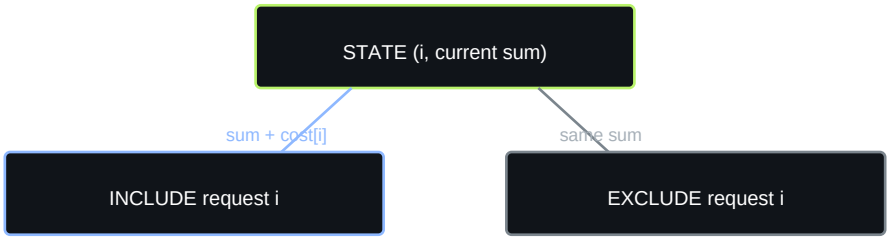
3. Algorithm explanation

At index i, the recursive search makes two decisions in order: INCLUDE request i, then EXCLUDE request i. The path maintains its exact integer sum and selected 1-based indices. When the sum equals T, the subset is recorded and the branch ends; with strictly positive costs, no extension can also equal T.

The primary pass stores only the exact valid count, current minimum size, optimal count, and best index list. It never retains every valid subset.

4. Include/exclude pseudocode

```
SEARCH(i, sum, chosen):
    if sum > T: return                      # safe over-target prune
    if sum == T: record(chosen); return
    if sum + suffix_sum[i] < T: return      # safe unreachable prune
    if i == n: return
    SEARCH(i + 1, sum + cost[i], chosen + [i+1])  # INCLUDE
    SEARCH(i + 1, sum, chosen)                    # EXCLUDE
```



5. Safe pruning

The over-target prune is safe because all future costs are positive, so a sum greater than T cannot decrease back to T. The suffix-sum prune is safe because even selecting every remaining request cannot reach T when current_sum + remaining_suffix_sum < T. Reaching T is terminal because all costs are strictly positive.

The implementation does not prune based on the current number of selected requests: such a branch may still contain valid subsets that contribute to COUNT.

6. Correctness proof

Completeness. For every request, a subset either contains it or does not. The recursion explores both branches, so every subset is represented by exactly one root-to-state decision path unless a safe prune proves it cannot be a target subset. Therefore every valid subset is visited and counted once.

Soundness. A subset is recorded only when the maintained sum of its selected input costs equals T, so every recorded subset is valid.

Minimum and optimal count. The search visits every valid subset. It replaces the best size and resets the optimal count when it finds a smaller valid subset; it increments the optimal count for every subset of equal best size. At completion these are exactly the global minimum and its count.

Lexicographic selection. Every minimum-size candidate is an increasing list of 1-based input indices. Explicit list comparison uses the first differing index, which is the defined lexicographic order; retaining the least candidate therefore returns the required tie-break result.

7. Complexity

Worst-case time is O(2^n), since each request may produce an include and exclude branch. Auxiliary recursion space is O(n), excluding the bounded optional trace. Safe pruning reduces work on many instances but does not change the worst-case time bound.

8. Sample and edge-case verification

Case	Input costs / target	COUNT · MIN · OPT · CHOSEN
Provided sample	[5,5,10,3,7,10] / 10	4 · 1 · 2 · 3
Impossible	[2,4,6] / 1	0 · -1 · 0 · NONE
Target zero	[1,2,3,4] / 0	1 · 0 · 1 · EMPTY
Duplicates	[2,2,3] / 4	1 · 2 · 1 · 1 2
Tie break	[2,1,1,2] / 3	4 · 2 · 4 · 1 2

Testing also covers unsorted requests, multiple minimum-size solutions, n=22, output formatting, and deterministic randomized small inputs checked against an independent brute-force oracle.

9. Python implementation

The following is the complete standalone source, identical to the generated CODE.py deliverable. It uses only the Python standard library, reads standard input, and writes only the required four output lines to standard output.

CODE.py · lines 1–42

```
"""Exact subset-sum solver using include/exclude recursive backtracking.

This module is also the standalone submission source. It intentionally uses no
non-standard dependencies and keeps the original request order throughout.
"""
from __future__ import annotations

import sys
import time
from typing import Any, Iterator

MAX_N = 22
MAX_TARGET = 1_000_000
MAX_COST = 1_000_000


def validate_problem(costs: list[int], target: int) -> None:
    if type(target) is not int or not 0 <= target <= MAX_TARGET:
        raise ValueError("Target must be an integer from 0 to 1,000,000.")
    if len(costs) > MAX_N:
        raise ValueError("At most 22 requests are allowed.")
    for index, cost in enumerate(costs, start=1):
        if type(cost) is not int or not 1 <= cost <= MAX_COST:
            raise ValueError(f"Request #{index} cost must be an integer from 1 to 1,000,000.")


def solve_problem(costs: list[int], target: int, trace_limit: int = 0) -> dict[str, Any]:
    """Count all exact subsets and optimal subsets in input-index order.

    Every visited state is reached by recursive include/exclude choices. The
    only pruning rules are over-target and insufficient remaining suffix sum.
    A bounded optional event list records genuine visited states.
    """
    validate_problem(costs, target)
    started = time.perf_counter()
    n = len(costs)
    suffix = [0] * (n + 1)
    for index in range(n - 1, -1, -1):
        suffix[index] = suffix[index + 1] + costs[index]

    count = 0
    min_size: int | None = None
```

CODE.py · lines 43–84

```
    optimal_count = 0
    best_indices: list[int] | None = None
    explored_nodes = 0
    pruned_nodes = 0
    events: list[dict[str, Any]] = []
    trace_cap = max(0, min(int(trace_limit), 5_000))

    def visit(index: int, current_sum: int, chosen: list[int], parent_id: int | None,
              decision: str, depth: int) -> None:
        nonlocal count, min_size, optimal_count, best_indices
        nonlocal explored_nodes, pruned_nodes

        explored_nodes += 1
        event_id = explored_nodes - 1
        status = "exploring"
        remaining = target - current_sum
        if current_sum > target:
            status = "pruned"
            pruned_nodes += 1
        elif current_sum == target:
            count += 1
            size = len(chosen)
            candidate = chosen.copy()
            if min_size is None or size < min_size:
                min_size = size
                optimal_count = 1
                best_indices = candidate
            elif size == min_size:
                optimal_count += 1
                if best_indices is None or candidate < best_indices:
                    best_indices = candidate
            status = "valid"
        elif current_sum + suffix[index] < target:
            status = "pruned"
            pruned_nodes += 1
        elif index >= n:
            status = "dead_end"

        if len(events) < trace_cap:
            events.append({
                "id": event_id,
                "parent_id": parent_id,
```

CODE.py · lines 85–126

```
                "index": index,
                "request_index": index + 1 if index < n else None,
                "decision": decision,
                "current_sum": current_sum,
                "selected_indices": chosen.copy(),
                "remaining_target": remaining,
                "depth": depth,
                "status": status,
            })

        if status in ("pruned", "valid", "dead_end"):
            return

        # Include first, then exclude. Costs are not sorted or deduplicated.
        chosen.append(index + 1)
        visit(index + 1, current_sum + costs[index], chosen, event_id, "INCLUDE", depth + 1)
        chosen.pop()
        visit(index + 1, current_sum, chosen, event_id, "EXCLUDE", depth + 1)

    visit(0, 0, [], None, "START", 0)

    if events and min_size is not None:
        for event in events:
            if event["status"] == "valid":
                event["status"] = "optimal" if len(event["selected_indices"]) == min_size else "valid"

    elapsed_ms = (time.perf_counter() - started) * 1000
    return {
        "count": count,
        "minimum_size": min_size if min_size is not None else -1,
        "optimal_count": optimal_count,
        "chosen_indices": best_indices,
        "execution": {
            "explored_nodes": explored_nodes,
            "pruned_nodes": pruned_nodes,
            "duration_ms": round(elapsed_ms, 3),
        },
        "events": events,
        "trace_truncated": bool(trace_cap and explored_nodes > len(events)),
    }


```

CODE.py · lines 127–168

```python
def iter_matching_plans(costs: list[int], target: int, minimum_size: int,
                        optimal_only: bool | None = None,
                        search_index: int | None = None) -> Iterator[list[int]]:
    """Yield exact plans in lexicographic index-list order without storing all."""
    validate_problem(costs, target)
    n = len(costs)
    suffix = [0] * (n + 1)
    for index in range(n - 1, -1, -1):
        suffix[index] = suffix[index + 1] + costs[index]
    chosen: list[int] = []

    def walk(index: int, current_sum: int) -> Iterator[list[int]]:
        if current_sum > target or current_sum + suffix[index] < target:
            return
        if current_sum == target:
            is_optimal = len(chosen) == minimum_size
            if optimal_only is None or is_optimal == optimal_only:
                if search_index is None or search_index in chosen:
                    yield chosen.copy()
            return
        if index >= n:
            return
        chosen.append(index + 1)
        yield from walk(index + 1, current_sum + costs[index])
        chosen.pop()
        yield from walk(index + 1, current_sum)

    yield from walk(0, 0)


def paginate_plans(costs: list[int], target: int, minimum_size: int,
                   page: int, page_size: int, optimal_only: bool | None = None,
                   search_index: int | None = None) -> dict[str, Any]:
    """Re-run the traversal, count exact matches, and retain only one page."""
    if page < 1:
        raise ValueError("Page must be at least 1.")
    if not 1 <= page_size <= 100:
        raise ValueError("Page size must be from 1 to 100.")
    start = (page - 1) * page_size
    end = start + page_size
    matches: list[list[int]] = []
    total = 0
```

CODE.py · lines 169–210

```
    for plan in iter_matching_plans(costs, target, minimum_size, optimal_only, search_index):
        if start <= total < end:
            matches.append(plan)
        total += 1
    return {
        "plans": matches,
        "page": page,
        "page_size": page_size,
        "total": total,
        "total_pages": (total + page_size - 1) // page_size,
        "order": "lexicographic",
    }


def format_output(result: dict[str, Any]) -> str:
    chosen = result["chosen_indices"]
    if chosen is None:
        chosen_text = "NONE"
    elif not chosen:
        chosen_text = "EMPTY"
    else:
        chosen_text = " ".join(str(index) for index in chosen)
    return "\n".join((
        f"COUNT {result['count']}",
        f"MIN_SIZE {result['minimum_size']}",
        f"OPTIMAL_COUNT {result['optimal_count']}",
        f"CHOSEN {chosen_text}",
    ))


def main() -> None:
    tokens = sys.stdin.read().split()
    if len(tokens) < 2:
        raise SystemExit("Expected n T followed by n request costs.")
    try:
        n = int(tokens[0])
        target = int(tokens[1])
        if n < 0 or n > MAX_N or len(tokens) != n + 2:
            raise ValueError("Input must contain n, T, and exactly n costs (0 <= n <= 22).")
        costs = [int(token) for token in tokens[2:]]
        print(format_output(solve_problem(costs, target)))
    except ValueError as error:
```

CODE.py · lines 211–215

```
211          raise SystemExit(str(error)) from error
212
213
214  if __name__ == "__main__":
215      main()
```

10. Conclusion

Recursive include/exclude search provides a transparent and complete solution for the specified n≤22 domain. Safe suffix and over-target pruning preserve correctness while reducing unnecessary exploration; explicit size counting and lexicographic comparison supply the requested smallest-team result.

FUNDWISE / Every combination matters. Find the smallest team.